DUOC UC
ESCUELA DE INFORMÁTICA Y TELECOMUNICACIONES

ESPECIFICACIÓN DE REQUISITOS DE SOFTWARE

Proyecto: Market De La Esquina
Módulo Público / Rol Cliente

Revisión: 2.0
Fecha: 17 de octubre de 2026
Asignatura: Desarrollo Fullstack II (DSY1104)

Especificación de Requisitos según estructura IEEE 830

Ficha del documento

Fecha	Revisión	Autor	Modificación
17-10-2026	2.0	Bairon Valencia (bairon1313)	Reorganización del ERS del módulo cliente según estructura IEEE 830.
17-10-2026	2.0	Kenneth Araya (Kenneth-Araya)	Estructura del proyecto, pages admin y despligueme del AWS
17-10-2026	2.0	Benjamin Palacios (Darkneto17)	Pruebas unitarias

Documento de referencia: ERS V2.0 – Módulo Cliente – Market De La Esquina.

Alcance: Especificación exclusiva del módulo público / rol cliente de la Fase 2 (aplicación React SPA).

Validación del documento: pendiente de revisión y aprobación por las partes.

Contenido

Ficha del documento

1. Introducción

   1.1 Propósito

   1.2 Ámbito del sistema

   1.3 Definiciones, acrónimos y abreviaturas

   1.4 Referencias

   1.5 Visión general del documento

2. Descripción general

   2.1 Perspectiva del producto

   2.2 Funciones del producto

   2.3 Características de los usuarios

   2.4 Restricciones

   2.5 Suposiciones y dependencias

   2.6 Requisitos futuros

3. Requisitos específicos

   3.1 Requisitos comunes de las interfaces

   3.1.1 Interfaces de usuario

   3.1.2 Interfaces de hardware

   3.1.3 Interfaces de software

   3.1.4 Interfaces de comunicación

   3.2 Requisitos funcionales

   3.3 Requisitos no funcionales

   3.3.1 Rendimiento

   3.3.2 Seguridad

   3.3.3 Fiabilidad

   3.3.4 Disponibilidad

   3.3.5 Mantenibilidad

   3.3.6 Portabilidad

   3.4 Otros requisitos y trazabilidad

1. Introducción

Este documento especifica los requisitos del software del módulo cliente (tienda pública) del proyecto "Market De La Esquina". La versión documentada corresponde a la Fase 2, en la que las maquetas estáticas HTML5/CSS/JavaScript fueron migradas a una aplicación web de página única (SPA) desarrollada con React y Bootstrap 5.3.

1.1 Propósito

Definir el alcance, contexto, arquitectura frontend y requisitos funcionales del módulo público para orientar su implementación, revisión y pruebas. Está dirigido al equipo de desarrollo, responsables del proyecto y evaluadores de la asignatura Desarrollo Fullstack II (DSY1104).

1.2 Ámbito del sistema

El módulo ofrece la experiencia pública del cliente de "Market De La Esquina": navegación, página principal, catálogo, carrito de compras, secciones Nosotros y Blogs, contacto, registro e inicio de sesión. El alcance de esta ERS se limita a las vistas y flujos descritos en el documento fuente.

El sistema permite visualizar productos, administrar unidades del carrito, calcular subtotales y total, enviar formularios de contacto, registrar cuentas y verificar credenciales para iniciar sesión. El documento fuente describe persistencia local mediante localStorage; no especifica integración con un backend para estos flujos.

1.3 Definiciones, acrónimos y abreviaturas

Término	Definición
ERS	Especificación de Requisitos de Software.
SPA	Single Page Application; aplicación web de página única.
RF	Requisito funcional.
RUN	Rol Único Nacional, utilizado en el formulario de registro según las reglas descritas.
Navbar	Barra de navegación superior de la aplicación.
localStorage	Mecanismo del navegador utilizado por el módulo para guardar datos localmente.
CSS	Hojas de estilo en cascada utilizadas para presentación visual.

1.4 Referencias

- Documento fuente: ERS_Cliente_Market_De_La_Esquina_V3, versión 2.0, actualizado el 17 de octubre de 2026.
- Repositorio indicado en el documento fuente: react-market-dela-esquina.
- Estructura documental de referencia: Anexo 4 ERS – Especificación de Requisitos del Software, estándar IEEE 830, proporcionado como plantilla.

1.5 Visión general del documento

La sección 2 presenta el contexto del módulo, sus funciones, perfiles de usuario, restricciones y dependencias. La sección 3 detalla las interfaces, requisitos funcionales identificados como RF-CLI-01 a RF-CLI-10, requisitos no funcionales documentados y trazabilidad del desarrollo.

2. Descripción general

El módulo cliente se organiza en componentes independientes, con estado local administrado mediante useState, efectos de ciclo de vida mediante useEffect y hojas de estilo separadas por vista. La navegación entre vistas se realiza con React Router DOM.

2.1 Perspectiva del producto

El módulo forma parte del proyecto web Minimarket Online Market De La Esquina. La arquitectura descrita en la fuente está basada en React 18, React Router DOM, Bootstrap 5.3, CSS modular y almacenamiento local del navegador. PublicLayout.jsx actúa como contenedor compartido de las vistas públicas.

2.2 Funciones del producto

- Proveer navegación global y pie de página para las vistas públicas.
- Mostrar el banner principal y productos destacados, sincronizados con el catálogo.
- Presentar el catálogo dinámico con tarjetas de productos y controles de cantidad.
- Permitir revisar el carrito, modificar cantidades, eliminar productos y calcular importes.
- Mostrar información institucional y publicaciones de comunidad.
- Permitir enviar consultas, sugerencias o reclamos mediante un formulario validado.
- Registrar usuarios y validar el inicio de sesión con cuentas guardadas localmente.
- Actualizar el contador del carrito y el menú de perfil según el estado guardado.

2.3 Características de los usuarios

Cliente visitante: navega por la tienda, revisa el catálogo, utiliza el carrito y accede a las secciones informativas y de contacto.

Cliente registrado: puede completar el inicio de sesión con una cuenta previamente almacenada y visualizar el menú de perfil correspondiente.

El documento fuente no especifica requisitos de formación, experiencia técnica ni perfiles administrativos para este alcance; esta ERS cubre únicamente el módulo público / rol cliente.

2.4 Restricciones

- Stack documentado: React 18, React Router DOM, Bootstrap 5.3, CSS modular y JavaScript.
- El módulo utiliza localStorage para persistir carrito, usuarios registrados, sesión y formularios de contacto.
- Las vistas y rutas descritas incluyen /, /productos, /nosotros, /blogs, /contacto, /carrito, /registro y /login.
- El diseño debe mantener hojas de estilo separadas por vista para evitar colisiones visuales.
- El registro contempla reglas específicas para RUN chileno, selección de región/comuna, dominios de correo permitidos y longitud de contraseña.

2.5 Suposiciones y dependencias

El funcionamiento descrito depende de un navegador con soporte para localStorage y de la ejecución correcta de la aplicación React. La navegación depende de React Router DOM y la presentación visual utiliza Bootstrap 5.3 y CSS por componente. El documento fuente no define una base de datos remota ni servicios backend para las operaciones del módulo cliente.

2.6 Requisitos futuros

El documento fuente no define formalmente requisitos futuros. Cualquier ampliación —por ejemplo, persistencia en servidor, integración de pagos o administración remota de cuentas— queda fuera del alcance documentado y requeriría definición y aprobación posterior.

3. Requisitos específicos

Los requisitos siguientes se reorganizan desde el documento fuente y conservan sus identificadores RF-CLI-01 a RF-CLI-10, prioridades, componentes y reglas técnicas descritas.

3.1 Requisitos comunes de las interfaces

3.1.1 Interfaces de usuario

La interfaz corresponde a una aplicación web responsiva con navegación superior, área de contenido y pie de página. El documento fuente identifica vistas para inicio, productos, nosotros, blogs, contacto, carrito, registro e inicio de sesión. Se utiliza Bootstrap 5.3 y CSS modular por vista.

3.1.2 Interfaces de hardware

El documento fuente no establece requisitos específicos de hardware. El módulo se presenta como una aplicación web accesible mediante navegador.

3.1.3 Interfaces de software

Software / componente	Propósito documentado
React 18	Construcción de vistas y componentes funcionales.
React Router DOM	Navegación SPA entre rutas sin recarga completa.
Bootstrap 5.3	Estructura visual y diseño responsivo.
CSS modular	Estilos específicos de cada vista y prevención de colisiones visuales.
localStorage API	Persistencia local de carrito, usuarios, sesión y formularios.
GitHub	Control de versiones del repositorio react-market-dela-esquina.

3.1.4 Interfaces de comunicación

La sincronización entre vistas se describe mediante claves de localStorage y eventos personalizados: carritoActualizado y sesionActualizada. El documento fuente no especifica protocolos de comunicación con servicios externos.

3.2 Requisitos funcionales

RF-CLI-01: Configuración de PublicLayout con Navbar y Footer Globales

Atributo	Especificación
Componente asociado	src/layouts/PublicLayout.jsx
Prioridad	Alta (Esencial)
Descripción	El sistema debe proveer un contenedor global para todas las vistas del cliente que incorpore la barra de navegación superior con identidad de marca de Market De La Esquina y el pie de página corporativo en la parte inferior.
Reglas técnicas y de negocio	Navegación Centralizada: Provee enlaces directos mediante <Link> hacia las vistas de cliente: Home (/), Productos (/productos), Nosotros (/nosotros), Blogs (/blogs), Contacto (/contacto) y Carrito (/carrito). Renderizado Condicional en

	Vistas de Autenticación: Utilizando useLocation() , el layout detecta cuando el cliente se encuentra en /login o /registro y oculta la barra de navegación superior para dar protagonismo al diseño de autenticación a pantalla completa. Anclaje Estructural del Footer: Tanto en el layout general como en las vistas de Login y Registro, se implementa una estructura Flexbox vertical (d-flex flex-column justify-content-between min-vh-100) que asegura que el Footer institucional (con dirección en Av. Siempre Viva 742, Quilpué , teléfono y correo electrónico) quede fijo en el borde inferior de la ventana sin superponerse ni flotar sobre el contenido.

RF-CLI-02: Menú Desplegable de Perfil de Usuario y Contador Dinámico de Carrito

Atributo	Especificación
Componente asociado	src/layouts/PublicLayout.jsx
Prioridad	Alta
Descripción	El encabezado de navegación debe actualizar en tiempo real tanto el contador de artículos del carrito como el estado de la sesión del usuario sin requerir recargas de página.
Reglas técnicas y de negocio	Contador Dinámico de Carrito: Calcula el total de unidades almacenadas en carritoMarket y muestra una insignia numérica (#cart-badge) junto al icono del carrito. Escucha el evento carritoActualizado para reflejar instantáneamente cualquier suma o resta de productos hecha en el Home, en Productos o en el Carrito. Botones de Acceso vs. Perfil Activo: Cuando no existe sesión en usuarioActivo , muestra los botones estilizados de Iniciar Sesión y Registro . Cuando el usuario inicia sesión, estos botones se reemplazan automáticamente por el botón de perfil con su avatar circular y correo. Menú Desplegable de Perfil (Dropdown): Al hacer clic en el avatar del usuario autenticado, se despliega un menú flotante que exhibe la inicial del usuario, su correo electrónico y la acción "Cerrar sesión" , la cual limpia usuarioActivo de localStorage y redirige a /login .

RF-CLI-03: Migración de Home.jsx y Comunicación Estricta con Productos.jsx

Atributo	Especificación
Componente asociado	src/pages/client/Home.jsx , estilosHome.css y Bootstrap 5
Prioridad	Alta
Descripción	Transformación de la página principal estática (home.html) a un componente funcional reactivo (Home.jsx) que exhibe el banner promocional del minimarket y la sección de productos destacados (productosSection) sincronizada con el catálogo general.
Reglas técnicas y de negocio	Control Directo de Unidades en Home: Permite al cliente agregar productos al carrito y también sumar (+) o restar (-) unidades directamente desde las tarjetas de la página de inicio. Comunicación Estricta con Productos.jsx : El estado de las cantidades seleccionadas en Home.jsx lee y escribe directamente sobre la misma estructura de datos en localStorage (carritoMarket) utilizada por Productos.jsx . Si un cliente agrega 2 unidades de un artículo en el Home y navega a la pestaña Productos, la tarjeta de ese artículo refleja exactamente las mismas 2 unidades seleccionadas.

RF-CLI-04: Catálogo Dinámico de Productos con Control de Cantidades y CSS Modular

Atributo	Especificación

Componente asociado	src/pages/client/Productos.jsx y src/styles/productos.css
Prioridad	Alta
Descripción	El sistema debe presentar una vista dedicada al catálogo de comestibles de Market De La Esquina con renderizado dinámico de tarjetas, filtros visuales, paleta de colores corporativa corregida y controles interactivos de cantidad por ítem.
Reglas técnicas y de negocio	Renderizado de Catálogo Dinámico: Recorre la colección de productos mediante .map() mostrando imagen, nombre del artículo, categoría/descripción y precio unitario en pesos chilenos (CLP). Controles de Cantidad Integrados: Si un producto aún no está en el carrito, muestra el botón de acción para agregarlo. Una vez agregado, permite incrementar o disminuir sus unidades en tiempo real, eliminándolo de la lista si la cantidad baja a cero. Estética y CSS Modular (productos.css): Aplica correcciones de contraste y colores armónicos con la identidad del minimarket, garantizando tarjetas uniformes en cualquier resolución de pantalla.

RF-CLI-05: Vista de Carrito con Persistencia en LocalStorage y Control de Unidades

Atributo	Especificación
Componente asociado	src/pages/client/Carrito.jsx
Prioridad	Alta
Descripción	El sistema debe proveer una vista detallada donde el cliente pueda revisar todos los productos seleccionados, ajustar sus cantidades, eliminar artículos individuales o vaciar el carrito, con cálculo automático de importes.
Reglas técnicas y de negocio	Recuperación Persistente: Al montar el componente, carga automáticamente los artículos guardados en localStorage.getItem('carritoMarket') . Operaciones de Unidades (CRUD en Carrito): Dispone de botones para aumentar (+) o disminuir (-) la cantidad de cada producto y botón para eliminar el ítem por completo. Cálculo Automático de Subtotales y Total: Multiplica dinámicamente precio * cantidad por cada línea de producto y calcula la sumatoria general del pedido, actualizando simultáneamente el contador del PublicLayout .

RF-CLI-06: Vista Institucional "Nosotros" con Diseño Armónico y Logo Actualizado

Atributo	Especificación
Componente asociado	src/pages/client/Nosotros.jsx + CSS optimizado
Prioridad	Media
Descripción	Presenta la identidad, historia, misión y valores de cercanía de Market De La Esquina . Incorpora la actualización del logotipo institucional oficial de la tienda, maquetación moderna y responsiva en Bootstrap y optimización de estilos CSS para una lectura limpia y profesional.
Reglas técnicas y de negocio	Presenta la identidad, historia, misión y valores de cercanía de Market De La Esquina . Incorpora la actualización del logotipo institucional oficial de la tienda, maquetación moderna y responsiva en Bootstrap y optimización de estilos CSS para una lectura limpia y profesional.

RF-CLI-07: Módulo de "Blogs" con Tipografía Elegante y CSS Modular

Atributo	Especificación
Componente asociado	src/pages/client/Blogs.jsx + CSS modular de Blogs

Prioridad	Media
Descripción	Despliega artículos y publicaciones de interés para los clientes del minimarket (consejos, novedades y comunidad). Cuenta con una hoja de estilos CSS modular propia y selección de tipografía elegante que jerarquiza títulos, resúmenes e imágenes de cada entrada del blog.
Reglas técnicas y de negocio	Despliega artículos y publicaciones de interés para los clientes del minimarket (consejos, novedades y comunidad). Cuenta con una hoja de estilos CSS modular propia y selección de tipografía elegante que jerarquiza títulos, resúmenes e imágenes de cada entrada del blog.

RF-CLI-08: Formulario de Contacto Controlado con Validaciones, Alertas y LocalStorage

Atributo	Especificación
Componente asociado	src/pages/client/Contacto.jsx + CSS modular de Contacto
Prioridad	Alta
Descripción	El sistema debe permitir a los usuarios enviar consultas, sugerencias o reclamos mediante un formulario completamente controlado por React que valide la integridad de los datos antes de almacenarlos.
Reglas técnicas y de negocio	Componentes Controlados: Cada campo de entrada (Nombre, Correo electrónico, Asunto/Mensaje) está vinculado a su propio estado mediante useState . Validaciones de Formulario y Corrección de Reglas: Verifica que no existan campos vacíos al enviar, valida el formato y dominio del correo electrónico y controla la longitud permitida del texto. Alertas Dinámicas y Persistencia: Si ocurre un error de validación, despliega una alerta visual indicando el campo a corregir. Si el envío es válido, guarda el registro del mensaje en localStorage , muestra una alerta de confirmación exitosa y limpia los campos del formulario.

RF-CLI-09: Registro de Usuarios con Validaciones Chilenas y Guardado en LocalStorage

Atributo	Especificación
Componente asociado	src/pages/client/Registro.jsx y src/styles/estilosLogin.css
Prioridad	Alta (Crítico)
Descripción	El sistema debe permitir el registro de nuevos usuarios validando exhaustivamente sus datos personales, geográficos y de seguridad, almacenando la cuenta en localStorage para su posterior verificación en el Login.
Reglas técnicas y de negocio	Validación de RUN: Exige entre 7 y 9 caracteres, sin puntos ni guion, validando cuerpo numérico y dígito verificador numérico o K . Selectores Dinámicos Región / Comuna: Al seleccionar "Región Metropolitana" o "Valparaíso" , el selector de Comuna se desbloquea y carga dinámicamente las comunas correspondientes (ej. Valparaíso, Viña del Mar, Quilpué). Restricción de Dominios y Contraseña: Valida que el correo (máx. 100 caracteres) pertenezca a @duoc.cl , @duocuc.cl , @profesor.duoc.cl o @gmail.com , y que la contraseña tenga entre 4 y 10 caracteres coincidiendo con su confirmación. Control de No Duplicidad y Persistencia: Verifica en usuariosRegistrados que el RUN o correo no estén repetidos. Al aprobar, guarda el objeto completo del usuario (incluyendo su contrasena) en localStorage y redirige a /login .

RF-CLI-10: Inicio de Sesión Verificado contra Usuarios Registrados en LocalStorage

Atributo	Especificación
Componente asociado	src/pages/client/Login.jsx y src/styles/estilosLogin.css
Prioridad	Alta (Crítico)
Descripción	El sistema debe autenticar a los clientes verificando no solo el formato del correo y clave, sino confirmando contra localStorage que el usuario verdaderamente haya realizado su registro previo.
Reglas técnicas y de negocio	Validación de Formato: Comprueba campos obligatorios, correo ≤ 100 caracteres con dominio permitido y contraseña de 4 a 10 caracteres. Confirmación Real contra Registro: Consulta la lista usuariosRegistrados en localStorage . Si el correo ingresado no existe en los registros, rechaza el acceso con el mensaje: "Este correo no está registrado. Por favor regístrate primero." Validación de Contraseña Real: Si el correo existe, compara la clave ingresada con la propiedad contrasena guardada durante el registro. Si no coinciden, muestra: "Contraseña incorrecta. Inténtalo nuevamente." Apertura de Sesión y Layout: Al coincidir las credenciales, almacena la sesión en usuarioActivo , emite el evento sesionActualizada para actualizar el Navbar con el menú desplegable de perfil y redirige al Home (/). Incluye además el Footer anclado en la parte inferior y el enlace hacia /registro .

3.3 Requisitos no funcionales

3.3.1 Requisitos de rendimiento

El documento fuente describe actualización del contador y sincronización del carrito en tiempo real, sin recarga de página. No establece umbrales medibles de tiempo de respuesta, concurrencia o transacciones por segundo.

3.3.2 Seguridad

El registro valida el RUN, los datos geográficos, el formato y dominio del correo y la longitud de la contraseña. El inicio de sesión verifica que el correo exista en usuariosRegistrados y compara la contraseña guardada. La fuente describe almacenamiento en localStorage; no especifica cifrado, protección de secretos ni controles de seguridad del lado del servidor.

3.3.3 Fiabilidad

La fuente contempla persistencia local de los datos y sincronización de las vistas mediante eventos. No define métricas de incidentes, recuperación ante fallos ni objetivos cuantitativos de fiabilidad.

3.3.4 Disponibilidad

El documento fuente no define un porcentaje objetivo de disponibilidad ni un acuerdo de nivel de servicio.

3.3.5 Mantenibilidad

La separación de componentes React y hojas CSS por vista busca reducir colisiones y facilitar el mantenimiento. La organización de carpetas indicada incluye pages/client, layouts, styles y assets.

3.3.6 Portabilidad

La solución se define como aplicación web construida con React y tecnologías frontend. El documento fuente no especifica una matriz de navegadores o sistemas operativos soportados.

3.4 Otros requisitos y trazabilidad

El documento fuente incorpora una cronología de commits del repositorio y una matriz de componentes del módulo. A continuación se conserva la trazabilidad resumida disponible.

Fecha	Autor en GitHub	Mensaje del Commit Registrado	Impacto Técnico en el Módulo Cliente (Requisitos Cubiertos)
13 Oct 2026	bairon1313 / Kenneth-Araya	"Se agregan las estructuras de Cliente y Administrador para dejar funcional las vistas correspondientes" / "Se crea Login con su estilo y se agregan assets"	Creación de la arquitectura base de carpetas (pages/client , layouts , styles , assets) para iniciar la migración a React.
14 Oct 2026	bairon1313	"Se transforma de manera eficiente el home.html a home.jsx incluyendo tambien el css del home y el boostrap"	Cumplimiento de RF-CLI-03 : Migración de la vitrina principal a JSX integrando Bootstrap 5 y estilosHome.css .
14 Oct 2026	bairon1313	"Se configura PublicLayout con Navbar y Footer globales para las vistas de cliente y tambien se arregla la funcion de productosSection"	Cumplimiento de RF-CLI-01 y RF-CLI-03 : Implementación de PublicLayout.jsx envolvente y renderizado funcional de productos en el Home.
14 Oct 2026	bairon1313	"Se implementa vista Nosotros con diseño armónico, moderno y CSS optimizado"	Cumplimiento de RF-CLI-06 : Construcción del componente Nosotros.jsx con maquetación responsiva y estilos dedicados.
14 Oct 2026	bairon1313	"se implementa vista de Contacto con estructura JSX y CSS modular"	Cumplimiento inicial de RF-CLI-08 : Maquetación en JSX y aislamiento de estilos para la vista Contacto.jsx .
15 Oct 2026	bairon1313	"se implementa vista Productos con catalogo dinamico, control de cantidades y CSS modular para productos"	Cumplimiento de RF-CLI-04 : Desarrollo de Productos.jsx con carga dinámica de artículos, suma/resta de unidades y productos.css .
15 Oct 2026	bairon1313	"se implementa Blogs.jsx, su CSS modular, tipografia elegante y correcciones de colores en productos.css"	Cumplimiento de RF-CLI-07 y mejora visual de RF-CLI-04 : Creación de la sección de Blogs y refinamiento cromático del catálogo.
15 Oct 2026	bairon1313	"se implementa vista de carrito con localStorage, controles de unidades, contador dinámico en PublicLayout y actualización de logo institucional en la sección Nosotros"	Cumplimiento de RF-CLI-02 , RF-CLI-05 y RF-CLI-06 : Carrito persistente en localStorage , badge numérico en tiempo real en el Navbar y nuevo logo en Nosotros.
15 Oct 2026	bairon1313	"se actualiza Contacto.jsx con componentes controlados, validaciones de formulario, alertas y persistencia en localStorage"	Cumplimiento total de RF-CLI-08 : Formularios controlados con useState , alertas de validación y guardado en localStorage .
16 Oct 2026	bairon1313	"Se arreglan detalles del home que impedia agregar productos y restar productos dejandolo estrictamente comunicado con el productos.jsx, tambien se solucionan problemas con las	Refinamiento de RF-CLI-03 , RF-CLI-04 y RF-CLI-08 : Sincronización bidireccional del estado del carrito entre Home y Productos, y ajuste de reglas en Contacto.

		validaciones de contacto"	
16 Oct 2026	bairon1313	"Se implementa la vista de Login con validaciones, persistencia y se actualiza PublicLayout con el menu desplegable de perfil"	Cumplimiento de RF-CLI-02 y RF-CLI-10 : Validación de dominios y clave en Login, sesión en usuarioActivo y dropdown de usuario con cierre de sesión en el Navbar.
16 Oct 2026	bairon1313	"Se corrigen inconvenientes en la vista del login y implementando el footer"	Cumplimiento de RF-CLI-01 y RF-CLI-10 : Ajuste Flexbox (min-vh-100) para anclar el Footer al borde inferior de la vista Login.
16 Oct 2026	bairon1313	"Se implementa localStorage para el registro y que de esta manera al hacer el login se confirme que verdaderamente el usuario hizo el registro"	Cumplimiento de RF-CLI-09 y RF-CLI-10 : Implementación completa de Registro.jsx y conexión con Login.jsx mediante usuariosRegistrados .
17 Oct 2026	bairon1313	"Estructura fase 2 lista y gitignore reparado desde cero" / "Elimina definitivamente node_modules de la raiz" / "Agrega los archivos de la fase 1 al repositorio"	Organización final del repositorio dividiendo limpiamente fase1 y fase2 , optimización de .gitignore y documentación en carpeta docs para la entrega.

Matriz de componentes del módulo cliente

Vista / Funcionalidad	Ruta SPA	Componente React y Estilos	Persistencia y Eventos Utilizados
Layout Público Global	Envolvente (/)	PublicLayout.jsx	Lee usuarioActivo y carritoMarket ; escucha sesionActualizada y carritoActualizado .
Página Principal (Home)	/	Home.jsx + estilosHome.css	Lee/Escribe carritoMarket sincronizado con Productos.jsx .
Catálogo de Productos	/productos	Productos.jsx + productos.css	Lee/Escribe carritoMarket ; emite carritoActualizado .
Carrito de Compras	/carrito	Carrito.jsx	CRUD de unidades sobre carritoMarket en localStorage .
Nosotros	/nosotros	Nosotros.jsx + CSS optimizado	Componente institucional con logo actualizado.
Blogs	/blogs	Blogs.jsx + CSS modular	Renderizado de artículos con tipografía y diseño modular.
Contacto	/contacto	Contacto.jsx + CSS modular	Estados controlados (useState), validaciones y persistencia en localStorage .
Registro de Usuario	/registro	Registro.jsx + estilosLogin.css	Guarda cuentas validadas en usuariosRegistrados (localStorage).
Inicio de Sesión (Login)	/login	Login.jsx + estilosLogin.css	Verifica cuenta en usuariosRegistrados y crea sesión en usuarioActivo .

ANEXO DE CAMBIOS – MÓDULO ADMINISTRATIVO

Proyecto: Market De La Esquina | Revisión 2.1

4. Resumen de los componentes administrativos revisados

Este anexo incorpora al ERS una descripción de los componentes administrativos compartidos para revisión. Se documenta únicamente el comportamiento visible en el código proporcionado; no se presume que exista persistencia remota, conexión con microservicios o datos reales cuando el código no lo demuestra.

Archivo / componente	Propósito	Resumen del comportamiento
Admin.jsx	Panel principal	Presenta tarjetas con cifras de productos, usuarios, ventas e ingresos; accesos rápidos a productos y usuarios; y una lista de actividad reciente. Los valores de las tarjetas y la actividad mostrada están definidos directamente en el componente.
ProductosAdmin.jsx	Resumen de productos del carrito	Obtiene los artículos desde CartContext, busca sus datos en el catálogo y calcula productos distintos, unidades y total. Muestra una tabla con producto, peso, precio unitario, cantidad y subtotal. Si el carrito está vacío, informa que no hay productos agregados.
ReportesAdmin.jsx	Reportes del negocio	Lee ventas y usuarios mediante la función leer(). Agrupa las ventas por producto, calcula unidades e ingresos por producto, identifica el producto con más unidades, suma los ingresos y agrupa usuarios por tipo.
UsuariosAdmin.jsx	Administración/consulta de usuarios	Lee usuarios mediante leer('usuarios', []). Presenta totales y conteos por tipo de usuario, además de una tabla con nombre, RUN, correo, tipo, región/comuna y estado. Incluye un enlace para registrar un usuario.
VentasAdmin.jsx	Historial de ventas	Lee las ventas mediante leer('ventas', []). Calcula ventas totales, ingresos, ticket promedio y fecha de la última venta. Presenta el detalle de cada venta con identificador, fecha, productos, total y estado.

Pruebas de AdminStarCards	Pruebas de tarjetas KPI	Las pruebas buscan verificar el indicador de carga, los valores y tendencias de las tarjetas de ventas, pedidos pendientes, clientes activos y tasa de conversión, además de una clase visual de alerta para pedidos pendientes.

5. Requisitos funcionales incorporados como cambios

Los siguientes requisitos se agregan como una ampliación documental del alcance original. Su inclusión describe el comportamiento esperado a partir de los componentes compartidos y no certifica que el módulo esté integrado con un backend.

ID	Nombre	Requisito	Estado / observación
RF-ADM-01	Panel administrativo	El sistema deberá mostrar un panel de administración con indicadores resumidos de productos, usuarios, ventas e ingresos, accesos rápidos a las secciones administrativas y una lista de actividad reciente.	Parcial: los indicadores y la actividad del código revisado son datos estáticos.
RF-ADM-02	Resumen de productos del carrito	El sistema deberá presentar los productos que estén en el carrito y calcular productos distintos, unidades totales, precio unitario y subtotal por artículo, además del total general.	Implementado en el componente mostrado; representa el carrito, no un catálogo de inventario administrativo.
RF-ADM-03	Reporte general	El sistema deberá calcular los ingresos totales y el número de ventas, identificar el producto con más unidades vendidas y mostrar la distribución de usuarios por tipo.	Depende de los datos que entregue leer('ventas', []) y leer('usuarios', []).
RF-ADM-04	Consulta de usuarios	El sistema deberá mostrar el listado de usuarios registrados con nombre, RUN, correo, tipo, región, comuna y estado, así como los conteos por tipo de	El código revisado consulta datos locales; no muestra operaciones para editar o eliminar usuarios.

		usuario.	
RF-ADM-05	Historial de ventas	El sistema deberá mostrar el número de ventas, ingresos acumulados, ticket promedio, fecha de la última venta y el detalle de cada transacción.	Depende de leer('ventas', []); el formato de fecha se presenta según la configuración es-CL del navegador.
RF-ADM-06	Estados vacíos	Cuando no existan productos en el carrito, usuarios registrados, ventas o datos suficientes para reportes, el sistema deberá mostrar un mensaje informativo en lugar de una tabla sin explicación.	Se observan mensajes de estado vacío en los componentes de productos, usuarios, ventas y reportes.
RF-ADM-07	Indicadores de tarjetas	Las tarjetas KPI deberán mostrar un estado de carga mientras se obtienen los datos y representar sus valores e indicadores de tendencia una vez disponibles.	Requisito derivado de las pruebas de AdminStarCards; debe comprobarse contra la implementación real del componente.
RF-ADM-08	Alerta de pedidos pendientes	La tarjeta de pedidos pendientes deberá aplicar un estilo visual de advertencia cuando la cantidad supere el umbral crítico definido por el sistema.	La prueba espera la clase border-warning, pero el umbral y el origen de datos no están definidos en el fragmento entregado.

6. Observaciones técnicas y asuntos pendientes

- Persistencia: ReportesAdmin, UsuariosAdmin y VentasAdmin leen datos a través de ../../utils/storage. Debe confirmarse qué hace internamente esa utilidad y si utiliza localStorage u otro mecanismo. En los fragmentos entregados no se observa una llamada directa a un backend.
- Datos de demostración: Admin.jsx muestra cifras fijas (24 productos, 12 usuarios, 37 ventas y $482k) y una actividad predefinida. Para que el panel sea consistente, estos valores deberían obtenerse de una fuente de datos real o marcarse explícitamente como datos de ejemplo.
- Alcance de ProductosAdmin: aunque el nombre sugiere administración del catálogo, el componente presentado resume los artículos del carrito del cliente. No implementa creación, edición, eliminación ni listado completo de productos del inventario.
- Pruebas de AdminStarCards: el fragmento mostrado comenta la importación de render, screen y waitFor, pero luego usa render y screen. Si el archivo está exactamente así, deben habilitarse/importarse esas funciones para que las pruebas puedan ejecutarse. También se debe verificar que el componente implemente los data-testid utilizados.

- Usuarios: el código presentado permite consultar usuarios y navegar al registro, pero no demuestra edición, eliminación, cambio de rol ni gestión de permisos.
- Reportes y ventas: el código asume estructuras como venta.productos, producto.cantidad, producto.subtotal, venta.total, venta.fecha y venta.estado. Se debe validar que el proceso de finalizar venta guarde esos campos de manera consistente.
- Seguridad: si los datos administrativos se conectan a servicios reales, el acceso a rutas y operaciones deberá validarse mediante autenticación y autorización en el backend. El fragmento no demuestra que estas protecciones estén implementadas.

7. Trazabilidad del cambio

Origen del cambio: fragmentos React entregados para Admin.jsx, ProductosAdmin.jsx, ReportesAdmin.jsx, UsuariosAdmin.jsx, VentasAdmin.jsx y pruebas de AdminStarCards. Fecha de incorporación documental: 17 de octubre de 2026. Este anexo amplía el ERS anterior sin eliminar los requisitos del módulo cliente.

Historial de revisiones

Versión	Cambio	Base
2.1	Se agrega anexo de componentes administrativos, requisitos RF-ADM-01 a RF-ADM-08 y observaciones técnicas pendientes.	Código React y pruebas compartidos en la conversación.